

CSC301

Design and Analysis of Algorithms

Growth Functions - Worked Examples

How to derive the growth class instead of memorizing it

Algorithm -> Count the work -> $C(n)$ -> Simplify -> Growth class

Use this sequence for every example in this handout.

COMSATS University Islamabad, Attock Campus

Lecture 7 - Asymptotic Analysis and Growth Rates

How to use this handout

1. Do not guess the complexity from the appearance of the code. First decide which operation you are counting.
2. Write the basic-operation count $C(n)$. The exact expression may contain constants and lower-order terms.
3. Simplify only after you understand where the count came from. Keep the dominant growth term.
4. State the tight growth rate using Θ when the exact long-run growth is known.

Example: $C(n) = n(n-1)/2 = (1/2)n^2 - (1/2)n \rightarrow$ dominant term $n^2 \rightarrow \Theta(n^2)$

Growth ladder

Pattern in the work	Typical count	Growth class
fixed amount of work	constant	$\Theta(1)$
repeated halving/doubling	$\log n$	$\Theta(\log n)$
one full pass	n	$\Theta(n)$
n passes, each logarithmic	$n \log n$	$\Theta(n \log n)$
all pairs / 2-D grid	n^2	$\Theta(n^2)$
3-D work / three full loops	n^3	$\Theta(n^3)$
two choices for every item	2^n	$\Theta(2^n)$
every possible ordering	$n!$	$\Theta(n!)$

1. Constant Growth - $\Theta(1)$

The amount of work stays fixed even when the input size n becomes larger.

Example 1 - Access one array element

```
x <- A[5]
```

- The statement executes once.
- Array size can be 10, 1,000, or 1,000,000; the number of accesses is still 1.
- Therefore $C(n) = 1$, which does not depend on n .

$C(n) = 1 \rightarrow \Theta(1)$

Example 2 - Swap two variables

```
temp <- a
a <- b
b <- temp
```

- There are exactly 3 assignments.

- $C(n) = 3$ for every input size.
- A fixed constant changes the amount of work, but not how it grows with n .

$$C(n) = 3 \rightarrow \Theta(1)$$

Example 3 - Sum using a direct formula

```
sum <- n(n+1)/2
```

- A fixed sequence of arithmetic operations is performed: addition, multiplication, and division.
- The number of operations does not increase when n increases.
- So the count remains constant.

$$C(n) = \text{constant} \rightarrow \Theta(1)$$

2. Logarithmic Growth - $\Theta(\log n)$

Logarithmic growth usually appears when the remaining problem is repeatedly cut by a constant factor.

Example 1 - Halving loop

```
while n > 1
  n <- n / 2
```

- For $n = 32$: $32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$. That is 5 divisions.
- After k divisions, the remaining value is $n / 2^k$.
- The loop stops when $n / 2^k = 1$, so $n = 2^k$ and $k = \log_2(n)$.

$$C(n) = \log_2(n) \rightarrow \Theta(\log n)$$

Example 2 - Doubling loop

```
i <- 1
while i < n
  i <- 2i
```

- For $n = 32$: $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 32$.
- After k iterations, $i = 2^k$.
- The loop stops when 2^k reaches n , so k is approximately $\log_2(n)$.

$$C(n) = \log_2(n) \rightarrow \Theta(\log n)$$

Example 3 - Binary Search

```
while low <= high
  mid <- (low + high) / 2
  keep only the relevant half
```

- Each comparison removes about half of the remaining search space.
- For 16 elements: 16 -> 8 -> 4 -> 2 -> 1.
- After k halvings, $n / 2^k = 1$, so $k = \log_2(n)$.

Worst-case comparisons grow as $\log_2(n) \rightarrow \Theta(\log n)$

3. Linear Growth - $\Theta(n)$

Linear growth appears when the amount of work increases in direct proportion to the number of input items.

Example 1 - Sum every array element

```
for i <- 1 to n
  total <- total + A[i]
```

- The addition executes once for every element.
- There are n elements, so there are n additions.
- Therefore $C(n) = n$.

$C(n) = n \rightarrow \Theta(n)$

Example 2 - Find the maximum

```
largest <- A[0]
for i <- 1 to n-1
  if A[i] > largest
    largest <- A[i]
```

- Choose the comparison $A[i] > \text{largest}$ as the basic operation.
- The comparison executes n-1 times.
- $C(n) = n-1$. The dominant term is n.

$C(n) = n - 1 \rightarrow \Theta(n)$

Example 3 - Linear Search in the worst case

```
for i <- 0 to n-1
  if A[i] = x
```

```
return i
```

- If x is the last element or is absent, every element is checked.
- That gives n comparisons in the worst case.
- So $C_{\text{worst}}(n) = n$.

$$C_{\text{worst}}(n) = n \rightarrow \Theta(n)$$

4. Linearithmic Growth - $\Theta(n \log n)$

This often appears when linear work is performed across logarithmically many levels, or when n items each require logarithmic work.

Example 1 - Outer linear loop with an inner halving loop

```
for i <- 1 to n
  j <- n
  while j > 1
    j <- j / 2
```

- The outer loop executes n times.
- For each outer iteration, the inner loop halves j until it reaches 1, so it executes $\log_2(n)$ times.
- Multiply the independent repetition counts: $n \times \log_2(n)$.

$$C(n) = n \log_2(n) \rightarrow \Theta(n \log n)$$

Example 2 - Merge Sort level-count view

- At one level of Merge Sort, all n elements are processed during merging.
- The problem size is halved at each level: $n, n/2, n/4, \dots, 1$.
- The number of levels is $\log_2(n)$.
- Linear work per level $\times$ logarithmic number of levels gives $n \log_2(n)$.

$$C(n) \text{ is proportional to } n \log_2(n) \rightarrow \Theta(n \log n)$$

Example 3 - Heap Sort work pattern

- There are approximately n removals/extractions.
- A removal may move down through a heap whose height is about $\log_2(n)$.
- Thus the total work is approximately $n \times \log_2(n)$.

$$C(n) \text{ is proportional to } n \log n \rightarrow \Theta(n \log n)$$

5. Quadratic Growth - $\Theta(n^2)$

Quadratic growth commonly appears when work is performed for every pair, or across an n by n grid.

Example 1 - Two complete nested loops

```
for i <- 1 to n
  for j <- 1 to n
    operation
```

- For each value of i , the inner operation executes n times.
- The outer loop itself executes n times.
- Therefore the total number of executions is $n \times n$.

$$C(n) = n \times n = n^2 \rightarrow \Theta(n^2)$$

Example 2 - Add two n by n matrices

```
for i <- 1 to n
  for j <- 1 to n
    C[i][j] <- A[i][j] + B[i][j]
```

- An n by n matrix contains n^2 cells.
- One addition is performed for each cell.
- Therefore there are n^2 additions.

$$C(n) = n^2 \rightarrow \Theta(n^2)$$

Example 3 - Compare every unordered pair

```
for i <- 1 to n-1
  for j <- i+1 to n
    compare A[i], A[j]
```

- The comparison counts are $(n-1) + (n-2) + \dots + 2 + 1$.
- This sum equals $n(n-1)/2$.
- Expand: $C(n) = (1/2)n^2 - (1/2)n$.
- The n^2 term dominates for large n .

$$C(n) = n(n-1)/2 \rightarrow \Theta(n^2)$$

6. Cubic Growth - $\Theta(n^3)$

Cubic growth appears when there are three independent n -sized dimensions of work.

Example 1 - Three complete nested loops

```
for i <- 1 to n
  for j <- 1 to n
    for k <- 1 to n
      operation
```

- The innermost operation executes n times for each pair (i,j) .
- There are n choices for i , n for j , and n for k .
- Multiply the three counts: $n \times n \times n$.

$$C(n) = n^3 \rightarrow \Theta(n^3)$$

Example 2 - Standard matrix multiplication

- The result matrix C has n^2 entries.
- To compute one entry $C[i][j]$, we combine n products.
- Therefore total scalar work is $n^2 \times n = n^3$.

$$C(n) \text{ is proportional to } n^3 \rightarrow \Theta(n^3)$$

Example 3 - Floyd-Warshall loop structure

```
for k <- 1 to n
  for i <- 1 to n
    for j <- 1 to n
      update distance[i][j]
```

- Each of the three loops executes n times.
- The update is therefore executed $n \times n \times n$ times.
- That gives n^3 updates.

$$C(n) = n^3 \rightarrow \Theta(n^3)$$

7. Exponential Growth - $\Theta(2^n)$

Exponential growth often appears when every input item independently creates two possible choices.

Example 1 - Generate all subsets

- Each element has two choices: included or not included.

- For 1 element there are 2 possibilities; for 2 elements, $2 \times 2 = 4$; for 3 elements, $2 \times 2 \times 2 = 8$.
- For n elements, multiply 2 by itself n times.

$$\text{Number of subsets} = 2^n \rightarrow \Theta(2^n)$$

Example 2 - Brute-force 0/1 Knapsack

- Each item is either taken or left out.
- Those are two independent choices for every one of the n items.
- A straightforward exhaustive algorithm may therefore inspect 2^n possible selections.

$$\text{Number of candidate selections} = 2^n \rightarrow \Theta(2^n)$$

Example 3 - All Boolean assignments

- Each Boolean variable can be True or False.
- With n variables, the number of assignments is $2 \times 2 \times \dots \times 2$, with n factors.
- Therefore exhaustive checking considers 2^n assignments.

$$\text{Number of assignments} = 2^n \rightarrow \Theta(2^n)$$

8. Factorial Growth - $\Theta(n!)$

Factorial growth appears when an algorithm tries every possible ordering of n distinct objects.

Example 1 - Generate all permutations

- There are n choices for the first position.
- After choosing one item, $n-1$ choices remain for the second position, then $n-2$, and so on.
- Multiply the choices: $n(n-1)(n-2)\dots 1$.

Number of orderings = $n!$ $\rightarrow \Theta(n!)$

Example 2 - Brute-force Travelling Salesman

- A brute-force approach tries possible visiting orders of the cities.
- The number of possible orders grows factorially because each next position has one fewer city available.
- Fixing a starting city changes the exact constant/form, but not the factorial growth nature.

Candidate tour orderings grow factorially $\rightarrow \Theta(n!)$

Example 3 - Try every job schedule ordering

- For n distinct jobs, choose one of n jobs first.
- Then choose one of $n-1$ remaining jobs, then $n-2$, and so on.
- The total number of schedules is $n!$.

Number of schedules = $n!$ $\rightarrow \Theta(n!)$

Final recognition guide

Use this table only after you can explain where the count comes from.

What the algorithm does	How the count is formed	C(n)	Growth
fixed work	same number of operations for every n	c	$\Theta(1)$
repeatedly halves/doubles	solve $n/2^k = 1$ or $2^k = n$	$\log n$	$\Theta(\log n)$
one complete pass	one operation per item	n	$\Theta(n)$
n times, each logarithmic	$n \times \log n$	$n \log n$	$\Theta(n \log n)$
all pairs / n by n	$n \times n$ or $n(n-1)/2$	n^2 dominant	$\Theta(n^2)$
three n-sized dimensions	$n \times n \times n$	n^3	$\Theta(n^3)$
two choices per item	$2 \times 2 \times \dots \times 2$	2^n	$\Theta(2^n)$
every ordering	$n(n-1)\dots 1$	$n!$	$\Theta(n!)$

Do not memorize the label first. Derive C(n), simplify it, then state the growth class.

Quick practice

1. A loop performs $n-1$ comparisons. What is C(n), what is the dominant term, and what growth class results?
2. An algorithm performs $n(n+1)/2$ additions. Expand the expression and identify the growth class.
3. A variable starts at 1 and doubles until it reaches n. Derive the number of iterations.
4. An outer loop executes n times and an inner loop halves a variable each time. Derive the total count.
5. An algorithm tries include/exclude for each of n items. Explain why the number of candidate solutions is 2^n .
6. An algorithm tries every possible ordering of n objects. Explain why the count is $n!$.